

Apresentação

No primeiro tópico vamos conhecer a disciplina de Projeto e Análise de Algoritmos e entender como será a dinâmica do curso, além de conhecer a bibliografia, ementa e objetivos.



Objetivo

Os alunos deverão aprender sobre conceito básicos de Complexidade de Algoritmos e reconhecer a importância da disciplina para sua formação.

- Compreender como a disciplina de Projeto e Análise de Algoritmos será útil;
- Conhecer as bibliografias Básicas e Complementares;
- Aprender o que será ensinado ao longo do semestre;
- Fixar os conceitos básicos sobre Complexidade de Algoritmos



Referência para Estudo

CORMEN, Thomas H. et al. Algoritmos: teoria e prática. 4. ed. Rio de Janeiro: LTC, 2024. **Parte IV**

CORMEN, Thomas H. Desmistificando algoritmos. Rio de Janeiro: Elsevier, 2014. **Capítulo 2**

DASGUPTA, Sanjoy; PAPADIMITRIOU, Christos H; VAZIRANI, Umesh Virkumar. Algoritmos. São Paulo: McGraw-Hill, 2009. **Capítulo 2**



[Plano de Ensino e Introdução](#)



[Introdução - Exercícios](#)

A) Elabore o melhor algoritmo para receber uma sequência de n números inteiros. Depois o algoritmo deve receber um número m e deve trazer como saída o número de vezes que o valor m apareceu nesta sequência.

- Considere $n < 1.000.000$
- NOTA: existe alguma consideração diferente caso m seja um inteiro entre 0 e 10.000, ou um inteiro entre 0 e 1.000.000.000.000?
- OBS: e se o valor de m fosse informado antes da sequência de n números?

B) Considere que cada operação leva 1ns em média em um determinado processador. Determine o tempo das funções $\log_2 n$, n , $3n$, $n \log_2 n$, n^2 , 2^n e $n!$ para as seguintes quantidades de operações: 10, 100, 1.000, 10.000, 100.000 e 1.000.000.



[Programação básica](#)

✓ Done



[Análise de Complexidade](#)

✓ Done



[Análise de Complexidade - Exercícios](#)

A) Implemente as funções de ordenação analisadas e faça um quadro comparativo do tempo de execução para ordenar:

- ($n=$)100.000 números aleatórios
- ($n=$)100.000 números já em ordem crescente

B) Elabore os seguintes algoritmos e analise o seu tempo de execução para diferentes entradas e determine a sua complexidade de tempo:

- Receber como parâmetro dois valores inteiros a e b e calcular a^b .
- Receber duas matrizes quadradas de mesma ordem ($n \times n$) e realizar a multiplicação entre elas.

?

[Avaliação Tópico 1](#)

✓ Done ▾

Questões finais do tópico 1 (AT₁)[Trabalho Prático 1 \(TP-1\)](#)

✓ Done ▾

Trabalho Prático 1 – Algoritmo de ordenação simples (BubbleSort)

O objetivo deste trabalho é permitir ao aluno implementar um algoritmo de ordenação visto em aula e apresentar uma [avaliação](#) numérica do desempenho em diferentes casos de teste.

Descrição do Trabalho:

O trabalho consiste em implementar e comparar o tempo de execução (em número de operações) do algoritmo de ordenação BubbleSort visto em sala. O mesmo deve ser capaz de ordenar diversos vetores de números inteiros. A entrada do programa é o nome de um arquivo texto contendo vetores de diferentes tamanhos. O conteúdo dos arquivos de teste são valores inteiros (um valor por linha). A última linha do arquivo é uma linha em branco indicando que terminou a lista numérica. Ou seja, não é apresentado a priori a quantidade de valores. Os arquivos de teste (usado pelo professor para [avaliação](#) do trabalho) podem conter 100, 1000, 10.000 ou 100.000 valores.

Serão testados três diferentes tipos de vetores: aleatórios, ordenados em ordem crescente, e ordenados em ordem decrescente. Você deve criar arquivos neste padrão para testar sua implementação.

A saída do programa NÃO é a lista de valores ordenados. A saída esperada do programa é um valor numérico informando a quantidade de operações realizadas para ordenar o vetor.

Exemplo:

Seu programa receberá como parâmetro o nome do arquivo e retornará como saída (na tela/terminal) a quantidade de operações realizadas para ordenar o vetor. A seguir alguns exemplos de saída:



```
> bubblesort ordenado_100.txt
```

```
Número de operações: 99
```

```
> bubblesort randomico_1000.txt
```

```
Número de operações: 855627
```

```
> bubblesort decrescente_10000.txt
```

```
Número de operações: 49995000
```

[Jupyter \[IDE online\]](#)[Kaggle \[IDE online\]](#)[Google Colab \[IDE online\]](#)

 [Visual Studio Code \[VSCode online\]](#)

 [WinLibs standalone build of GCC and MinGW-w64 for Windows](#)

 [Complexidade de diferentes algoritmos](#)

